

Comparing Rule-Based, Game-Theoretic, and Reinforcement-Learned Response Selection for Residential IoT Defense: A Controlled Mininet Evaluation with Disclosed Self-Correction

Author Name*

*Affiliation

Email: author@example.com

Abstract—Automated response selection for residential IoT networks is typically evaluated either as a detection problem (is the traffic malicious?) or through anecdotal demonstration, rarely as a controlled comparison of how the response itself is chosen. We present a registry-driven defense system for a Mininet-simulated residential IoT network that compares three independent response-selection policies – a rule-based classifier, a Stackelberg game-theoretic solver, and a PPO-trained reinforcement-learning agent – against two naive baselines and an external signature-based IDS (Suricata), on real captured traffic across 17 threat conditions (16 attack types plus benign). Our evaluation harness runs 255 real Mininet trials (1,275 recorded outcomes), reporting Wilson-score confidence intervals rather than point estimates, and includes ablations on the Stackelberg payoff table, PPO reward shaping, training-seed robustness, and decision-boundary policy divergence. Two findings distinguish this evaluation from a typical system paper. First, re-running the harness at three times the original sample size revealed that several headline numbers were optimistic at low N (aggregate detection accuracy fell from 90.0% to 73.7% [68.0, 78.8]); we report the revision rather than the more favorable number. Second, an attempt to strengthen an adaptive-attacker finding (source-IP rotation evading identity-based blocking) instead surfaced a real bug in our own evaluation module and, once fixed, failed to reproduce the original claim across 40 further trials – we report this as a correction, not a success. We conclude that the three response-selection policies are behaviorally indistinguishable on in-distribution traffic but diverge substantially near decision boundaries, that our system materially outperforms naive baselines with a wide, honestly-reported margin, and that every result in this paper is scoped to a single controlled VM and Mininet topology – external validity remains an explicitly open problem.

Index Terms—IoT security, intrusion response, Stackelberg games, reinforcement learning, Mininet, reproducibility

I. Introduction

Consumer and residential IoT devices are widely deployed with weak or absent built-in security, and network-level defense – detecting an attack and then choosing what to do about it – is increasingly delegated to automated systems. The detection side of this problem is well studied: signature-based tools such as Snort and

Suricata, and a large body of machine-learning intrusion-detection literature, address the question of whether observed traffic is malicious. The response-selection side – given a detected threat, which of several possible actions (alert, isolate, deceive, rate-limit, quarantine, ...) should actually be taken – receives comparatively little controlled, comparative evaluation. Individual mechanisms (game-theoretic defense, reinforcement-learned defense, moving-target defense) are each well represented in the literature, but rarely compared against each other, against simple baselines, and against an external detector, on the same captured traffic, in the same controlled environment.

This paper makes four contributions:

- 1) A registry-driven defense architecture for a simulated residential IoT network (Section III) in which a single declarative record per attack type drives traffic generation, detection, Stackelberg payoff lookup, PPO observation encoding, and dataset generation, so that adding a new attack type or defense action does not require touching each subsystem by hand.
- 2) A repeatable benchmark harness (Section IV) that runs three independent response-selection policies – rule-based, Stackelberg game-theoretic, and PPO reinforcement-learned – alongside two naive baselines and an external, real signature-based IDS, against the same captured Mininet traffic per trial, with every chosen action actually executed and independently verified rather than assumed successful.
- 3) A set of ablations and robustness checks (Section V) largely absent from comparable systems work: Wilson-score confidence intervals on every reported rate, a payoff-table sensitivity ablation, a reward-shaping ablation, a ten-seed PPO training-robustness sweep, and a direct test of policy divergence at synthetic decision boundaries rather than only on traffic that happens to classify confidently.

- 4) A transparent account of two self-corrections (Section VI): a threefold sample-size increase that revised several headline numbers downward, and an adaptive-attacker scale-up that discovered a real bug in our own evaluation code and, once fixed, failed to reproduce an earlier finding – reported as a correction rather than suppressed.

We deliberately scope every claim in this paper to the environment it was measured in: a single Mininet topology on one resource-constrained virtual machine (Section VII). We do not claim external validity, and we treat the absence of that claim as a feature of a rigorous internal evaluation, not an oversight to be hidden.

II. Related Work

No published study was found comparing rule-based, Stackelberg game-theoretic, and reinforcement-learned response selection for IoT defense on a shared dataset; the closest work in the literature compares detection methods alone, or evaluates a single game-theoretic or learned defense mechanism on its own metrics and environment. We situate our design choices against this literature as context, not as a claimed head-to-head benchmark.

Signature-based detection. An experimental comparison of Suricata and Snort found Suricata generally ahead on detection accuracy, scalability, and resource efficiency (e.g. 100% vs. 85.7%/66.7% on two DNS-tunneling variants) [1], motivating our choice of Suricata as the external detection baseline (Section IV-C). Prior work evaluating signature-based tools against IoT botnet datasets (ISOT, IoT-23, Bot-IoT) documents the same structural limitation we independently observe: signature/threshold detectors miss traffic that does not match a known pattern [2].

Game-theoretic and deception-based defense. A stochastic-game honeypot moving-target-defense study reports substantial real engagement gains from deception over a static defense (attacker packet interactions up 920 vs. 42; scanner engagement extended 14.6 $\times$) [3] – a different environment and metric than our controlled, single-attacker Mininet trials, but directionally consistent with our own DECOY action’s measured value (Section V-B). A 2025 paper models collaborative IoT packet-sampling against DDoS as a Stackelberg game and derives a sampling-rate lower bound that deters an attacker [4] – the same game-theoretic framing we use, but for a continuous sampling-rate decision rather than a choice among discrete response actions, and evaluated analytically rather than against captured traffic.

Reinforcement learning for defense. Recent surveys document deep RL as an active, promising direction for IoT intrusion detection while flagging reproducibility and dataset-realism concerns across the field [5] – concerns we address directly by training and evaluating our PPO policy against real, verified Mininet outcomes and by reporting a seed-robustness sweep (Section V-D) rather

than a single trained checkpoint. A 2025 paper evaluates an RL-tuned moving-target-defense mutation strategy for edge IoT against DDoS specifically on attack success rate, defense latency, and CPU overhead [6] – the same category of cost-conscious framing our own cost/overhead analysis uses (Section V-C), for mutating network configuration rather than selecting among discrete response actions.

None of the above papers shares our dataset, environment, or exact decision space, which is precisely why our results are reported as an internally-valid comparison between the arms tested here, not translated into a claimed standing against any of them.

III. System Architecture

A. Registry-driven attack and response model

The system is organized around a single declarative registry (AttackScenario) with one record per attack type, holding its traffic generator, rule-based detector, registered “preferred” response action, and the confidence/score thresholds that action requires. Every dependent subsystem – the rule-based decision policy, the Stackelberg payoff lookup, PPO’s one-hot scenario encoding, and dataset generation for the machine-learning baseline – derives its behavior from this registry rather than encoding per-attack logic separately in each place. This was a deliberate response to an early failure mode in the project’s own development: adding one new attack type by hand across six independently-maintained files produced four distinct, reproduced bugs (a malformed traffic-generation script that hung indefinitely, an insufficient process-termination grace window, a privileged-port binding failure, and output-buffering interference with the network simulator’s own control channel). The registry refactor eliminates that failure class by construction for every attack added afterward.

B. Threat model and attack catalog

Table I summarizes the 16 registered attack types plus the benign (normal) condition, each with a distinct traffic signature and a registered preferred response. Signatures span packet rate, port diversity, payload size, traffic direction (several attacks, e.g. `data_exfiltration` and `dns_tunneling_exfiltration`, reverse the usual device-is-target direction), and, for `c2_beaconing`, inter-packet timing regularity – the only signature in the registry that is not rate-, volume-, or size-based. Every detector’s numeric threshold window was placed in a verified, unclaimed gap relative to every other detector’s window, confirmed both by a parametrized test sweeping every attack’s synthetic signature through the complete detector set and by repeated live Mininet runs under real timing variance.

C. Defense actions

Ten actions are available to all three policies, all real, verified mechanisms executed inside the Mininet network rather than simulated effects: `ALLOW` and

TABLE I
Registered threat conditions and preferred responses

Condition	Signature axis	Preferred
normal	– (benign baseline)	ALLOW
reconnaissance port scan	many ports, moderate rate	DECOY
DoS flood	one port, very high rate	ISOLATE
brute force	one port, sustained attempts	BLOCK _SOURCE
data exfiltration	reversed direction, large payload	ISOLATE
exploit payload injection	oversized single payload	DECOY
TCP SYN flood	refused connects, one port	ISOLATE
ICMP ping flood	high-volume padded ICMP	THROTTLE
slow-loris exhaustion	many held-open connections	THROTTLE
DNS amplification	oversized inbound UDP	BANDWIDTH _CAP
DNS tunneling exfiltration	reversed, many small UDP	DECOY
MQTT message flood	many completed TCP conns.	THROTTLE
firmware tampering	reversed, periodic oversized push	QUARANTINE
buffer overflow probe	sustained oversized payload, 1 conn.	RESET _SESSIONS
credential replay	repeated uniform small UDP	BLOCK _SOURCE
rogue config beacon	reversed, low-confidence pattern	FORENSIC _CAPTURE
C2 beaconing	periodic inter-arrival timing	ISOLATE

ALERT (no enforcement); ISOLATE (interface down, verified by polling link state); DECOY (a real TCP banner service with NAT redirection, verified by a real redirected connection); THROTTLE (a protocol-aware iptables hashlimit rate cap, verified against real traffic – an unthrottled attacker completed 56/56 connection attempts in 3s, the same traffic under a 2/s cap completed 7); BLOCK_SOURCE (a source-scoped iptables DROP rule, verified by polling the installed rule); QUARANTINE (default-deny with an allowlist for known-legitimate hosts); RESET_SESSIONS (real kernel socket destruction via `ss -K`); FORENSIC_CAPTURE (preserves the detection pipeline’s own already-captured evidence); and BANDWIDTH_CAP (a `tc tbf` egress cap on the attacker’s interface, verified at the receiver: 300/300 packets delivered uncapped vs. 20/300 under the cap).

D. Response-selection policies

Rule-based. A deterministic mapping from the classified attack type and its score/confidence to the registry’s preferred action, with dedicated handling for the benign and low-confidence cases.

Stackelberg. A leader-follower game solved per detected threat type, with a full, independently-reasoned payoff table (attacker and defender utility for every action, for every threat type, in `config/policies.yaml`) rather than a derived or auto-generated one. The solver’s own selection

was verified to agree with the registry’s declared preferred action for all 17 conditions.

PPO. A proximal-policy-optimization agent (Stable-Baselines3) over a ten-action discrete space matching DefenseAction’s declaration order, trained against a deterministic normalized security-context observation. The deployed recipe is `net_arch=[64,64]`, `ent_coef=0.01`, `n_steps=batch_size=170`, learning rate 0.001, 25,500 timesteps. An initial `net_arch=[32,32]` network reliably mis-routed the benign scenario to DECOY regardless of training length; doubling network capacity resolved this at the project’s development seed. Section V-D reports a formal, multi-seed robustness check of this recipe rather than relying on that single seed’s success.

IV. Evaluation Methodology

A. Harness design

Our benchmark harness runs N real Mininet trials for every registered condition. Each trial (i) generates real traffic for that condition (the same generators the live system uses) and captures it, preserving the raw packet capture for later offline analysis; (ii) classifies the capture with the attack-type-agnostic rule-based detector; (iii) evaluates five arms against the identical resulting security context – the rule-based, Stackelberg, and PPO policies, plus NaiveBlockAllBaseline (isolate any detected threat, the common “one fixed response to every alert” design) and AlwaysAllowBaseline (never intervene, the floor); and (iv) executes and independently verifies each distinct action chosen across the five arms exactly once, for real. One trial produces every arm’s result; no arm requires a separate live run.

B. Metrics

Detection accuracy: did the shared detector correctly classify the captured traffic (one result per trial, shared across arms, reported as a sanity check rather than a point of comparison). **Matches preferred action:** of the attack trials, how often an arm’s chosen action was the registry’s declared preferred action. **Verified response rate:** of all trials, how often an arm both chose the preferred action and that action’s real effect was independently confirmed – our headline metric, zero whenever the wrong action was chosen, only crediting a checked success. Every reported rate carries a 95% Wilson-score confidence interval, computed directly rather than via the normal approximation (which misbehaves at small n or extreme proportions).

C. External detection baseline

A second, independent arm runs offline Suricata analysis of the same captured packet captures, against two rulesets: ET-Open (real-world community threat signatures) and a small lab-tailored ruleset encoding the same shape knowledge our own rule-based detectors use, so Suricata is judged on a fair signature-matching version of that

TABLE II

Five-arm comparison, $N = 15$ trials/condition ($n = 255$ trials)

Arm	Detection acc.	Verified resp.	Matches pref.
Rule-based (ours)	73.7[68.0, 78.8]	68.2[62.3, 73.6]	74.2[68.3, 79.3]
Stackelberg (ours)	73.7[68.0, 78.8]	68.2[62.3, 73.6]	74.2[68.3, 79.3]
PPO (ours)	73.7[68.0, 78.8]	68.2[62.3, 73.6]	74.2[68.3, 79.3]
NaiveBlockAll	73.7[68.0, 78.8]	26.7[21.6, 32.4]	22.1[17.3, 27.8]
AlwaysAllow	73.7[68.0, 78.8]	5.9[3.6, 9.5]	0.0[0.0, 1.6]

TABLE III

External detection comparison, offline analysis of 255 pcaps

Detector	Accuracy	TPR	FPR
Suricata + ET-Open	67.8%	67.1%	20.0%
Suricata + lab rules	27.1%	22.5%	0.0%
Ours (rule-based)	73.7[68.0, 78.8]	–	0.0%

knowledge too, not only on a strawman of unrelated real-world malware signatures. Suricata is compared on detection axis only (accuracy, true-/false-positive rate against ground truth); it identifies traffic but does not select or execute a containment action.

D. Scale

This paper reports the harness at `trials_per_condition = 15` across the full 17-condition registry: 255 real Mininet trials, 1,275 recorded outcomes, on 2026-09-24 ($\approx 2\text{h}09\text{m}$ wall-clock). An earlier run at $N = 5$ (Section VI-A) is discussed explicitly because several of its headline numbers did not hold at the larger sample.

V. Results

A. Main comparison

Table II reports the five-arm comparison. The three real policies are statistically indistinguishable from each other on this dataset and substantially outperform both baselines.

All values are percentages with 95% Wilson CIs. Our system’s 68.2% verified-response rate is $2.6\times$ NaiveBlockAllBaseline’s 26.7% and $11.6\times$ AlwaysAllowBaseline’s 5.9% – a large, real margin, though smaller than an earlier, smaller-sample estimate ($3.4\times/13.7\times$ at $N = 5$), because our own numbers were revised down in the same $N = 15$ re-run that produced this table (Section VI-A), not because the baselines improved. Mean detection latency (27,778ms) is dominated by real packet capture, not decision-making (Section V-C).

B. External detection comparison

Table III compares our detector against Suricata under both rulesets. A real, disclosed gap in the lab-tailored ruleset’s own coverage (Section VI-C) explains its low reported accuracy; it is not a defect in our own detector, which independently reaches 73.7%.

TABLE IV

Stackelberg payoff-table sensitivity (2,550 solves)

Noise level	Match rate (95% CI)
$\pm 5\%$	96.1[94.0, 97.4]
$\pm 10\%$	88.0[85.1, 90.5]
$\pm 20\%$	78.8[75.3, 82.0]
$\pm 35\%$	64.5[60.5, 68.4]
$\pm 50\%$	61.0[56.8, 65.0]

C. Cost of comparing three policies

Timing 300 real, direct calls per policy (isolated from Mininet and packet capture) gives mean decision latencies of 0.054ms (rule-based), 0.464ms (Stackelberg), and 1.136ms (PPO). Running all three sequentially, as our comparison architecture does, costs 1.654ms total; the marginal overhead of comparing three policies versus deploying only the slowest one is 0.518ms – approximately 0.002% of the harness’s own 27.8s mean real detection latency. Comparing multiple policies is not this system’s bottleneck.

D. PPO training seed-robustness

A single trained checkpoint is a weak basis for a robustness claim. We trained 10 independent models at the deployed recipe, varying only the random seed (1, 2, 3, 7, 13, 17, 42, 55, 88, 99), and checked full convergence to every registered preferred action: 8/10 fully converge (Wilson 95% CI [49.0, 94.3]). Both failures made the same class of mistake – a false-positive-style error on benign traffic (one seed chose THROTTLE, one chose DECOY, instead of ALLOW) – while every seed converged correctly on every real attack scenario. The wide CI is reported honestly rather than rounded into overconfidence: this supports “most seeds converge fully; this specific instability mode recurs in roughly a fifth of runs,” not a tighter claim.

E. Ablations

Stackelberg payoff sensitivity. We perturbed every value in the deployed payoff table by random noise proportional to its own magnitude ($\pm 5\%$ to $\pm 50\%$, 30 trials/level, 2,550 solves total) and measured the fraction of solves still matching the registry’s declared action. Table IV shows a monotonic, graceful degradation: small realistic tuning error barely moves the outcome (the qualitative payoff structure is doing most of the work), while at $\pm 50\%$ noise over a third of solves diverge, confirming the specific values still carry real information.

PPO reward shaping. We trained the real environment under two reward configurations, holding the recipe and a five-seed sweep fixed: the deployed shaped reward (different magnitudes for different outcome severities, e.g. letting an attack through unopposed is penalized twice as hard as choosing a merely suboptimal defensive response) against a flat reward collapsing every correct outcome to

the same reward and every incorrect outcome to the same penalty. Both configurations reached full convergence on 5/5 seeds (Wilson CI [56.6, 100.0] for both). We report this honestly as inconclusive at this sample size, not as evidence that shaping is unnecessary: both intervals are wide and fully overlapping, and the more likely places shaping would matter – sample efficiency, or a longer-horizon task than our single-decision environment – were not measured here.

F. Decision divergence near boundaries

Because real captured traffic in our lab classifies far from any threshold every time, the three policies choosing identical actions on Table II is closer to a consistency check than a comparison. We separately tested all three policy classes directly against synthetic security contexts: (i) threat-score/confidence perturbed around each attack’s own registered decision threshold (± 0.03 to ± 0.15 , 784 contexts across the full registry), and (ii) entirely unregistered “novel” attack types no policy was tuned for (12 contexts). Near decision boundaries, rule-based diverges from both Stackelberg and PPO on roughly two-thirds of ambiguous contexts (all-three agreement 32.5%), while Stackelberg and PPO track each other closely (100% mutual agreement) even off the training distribution – a positive finding that the learned policy generalizes toward the same strategic reasoning the game solver encodes explicitly. On entirely unregistered attack types, rule-based and Stackelberg both degrade to a safe ALERT default every time; PPO instead selects ISOLATE regardless of the actual threat score or confidence supplied – a genuine, disclosed limitation: PPO has no mechanism for recognizing “I have never seen anything like this,” unlike the other two policies’ explicit fallback.

G. Detector robustness (evasion margin)

Perturbing every numeric feature in each attack’s own signature by -30% to $+30\%$ and re-classifying through the complete detector set measures brittleness versus real margin. Results are uneven, not uniformly reassuring: robust classification rates range from 28.6% (dns_amplification, firmware_tampering, buffer_overflow – a crowded region of the numeric feature space) to 100% (dos, brute_force, exfiltration). We are explicit that this measures measurement-noise tolerance, not adversarial evasion resistance: the perturbation is symmetric, random, and non-adaptive, and does not search for the nearest evasive point the way a real, threshold-aware adversary would (Section VII).

VI. Discussion: Two Self-Corrections

We report two findings in this section specifically because our first attempt at strengthening each one did not simply add more of the same evidence – it changed what we could honestly claim.

A. Sample size revealed optimism, not confirmed it

An earlier harness run at $N = 5$ trials/condition across 16 conditions reported aggregate detection accuracy of 90.0% and verified-response rate of 85.0%. Re-running at $N = 15$ across the full 17-condition registry (Table II) revised both down substantially (73.7%, 68.2%). Several individual conditions that read as a clean 100% at $N = 5$ settled meaningfully lower at $N = 15$ – exactly the outcome a single unlucky trial swinging a small-sample percentage by a full 20 points predicts, now confirmed directly rather than left as a caveat. Two conditions merit a worked explanation rather than a shrug. tcp_syn_flood showed 20.0% detection accuracy but 53.3% verified response – response higher than detection – which raw per-trial inspection explains exactly: of 15 trials, 3 were correctly labeled, 6 were misclassified as brute_force (failing both metrics, different preferred action), and 5 were misclassified as dos_flood, which happens to share tcp_syn_flood’s own preferred action (ISOLATE) and therefore still passes response verification despite the wrong label ($3 + 5 = 8/15 = 53.3\%$, exactly the reported figure). icmp_ping_flood’s 0% verified response rate is now confirmed as reproducible rather than small-sample noise: 0/15 at $N = 15$ versus 0/5 previously, with a CI ([0.0, 20.4]) that now excludes anything above roughly one-fifth.

B. An adaptive-attacker claim that did not survive scaling

A separate module tests whether identity-based containment (BLOCK_SOURCE) actually contains a persistent attacker across repeated attempts, in two modes: fixed (the same source every round, a control) and rotate (a fresh, not-yet-blocked source every round). An initial five-round pilot suggested a real finding: a second, fresh spoofed source (.121) reached the sensor and was independently detected and blocked on its first attempt, consistent with BLOCK_SOURCE’s identity-scoped mechanism (confirmed directly from its implementation) offering no protection against a rotating attacker.

We scaled this to 30 rounds/mode to obtain more than two data points. The scaled run did not produce the expected volume of clean evidence – instead, reading the raw per-round output revealed a real bug in our own module: rotate-mode’s source selection always chose the first not-yet-blocked address in a fixed pool. Because the second address’s capture came back empty (ambiguous between “blocked” and “not observed,” a distinction our own retry logic is designed to disclose rather than conflate), it stayed first-in-line and was re-selected for all 28 remaining rounds instead of the campaign rotating through the rest of the pool. We fixed the selection logic, added a regression test, expanded the source pool from 6 to 21 addresses, and re-ran (25 rounds/mode, genuinely cycling through 20 distinct spoofed addresses this time, confirmed from the log). Every one of the 20 distinct spoofed addresses produced zero captured packets, on

both the initial attempt and the built-in retry (40 real zero-packet observations), in two independent bug-fixed runs. Only the physical attacker’s own real, unspoofed address ever produced capturable traffic.

The honest conclusion this now supports is weaker than the original pilot suggested: the one successful spoofed capture that grounded our original claim could not be reproduced across 40 further attempts. We report this as one unreplicated pilot observation, not confirmed by two larger, bug-fixed follow-up attempts – not as strengthened evidence for source-rotation evasion. The architectural argument (BLOCK_SOURCE is scoped to the packet’s claimed source address, not the physical sender) still stands independent of this measurement, but we no longer claim repeated Mininet evidence that a rotating attacker gets through. Root-causing why spoofed traffic is not reliably captured in this environment (host-level reverse-path filtering was checked and does not explain it; a per-namespace or virtual-switch-level cause was not further isolated) is left as future work.

VII. Limitations

Single environment. Every result in this paper comes from one Mininet topology on one resource-constrained (2-CPU, $\approx$ 2.9 GB) virtual machine. No claim here should be read as generalizing to arbitrary IoT hardware, network topologies, or attacker populations; we have not tested beyond this single environment.

No adaptive, threshold-aware attacker. Our robustness measurements (Section V-F) use symmetric, non-adaptive perturbation. A real adversary that probes and adapts to our specific detection thresholds was not modeled.

External detector comparison is partial. The lab-tailored Suricata ruleset covers only 5 of 17 registered attacks, written when the registry was smaller and never extended; one of those five (dos_flood) does not reliably fire despite its port, protocol, and target matching the real traffic generator exactly – a real, disclosed gap in this specific comparison arm, found while regenerating results for this paper, not fixed here.

Shared detection across internal arms. The three policies and two baselines receive identical detection results per trial; our comparison measures response selection and execution, not detection accuracy, which is why the Suricata arm exists as an independent check.

Adaptive-attacker evidence is now explicitly weak, as detailed in Section VI-B.

VIII. Ethics and Responsible Disclosure

Every attack technique implemented in this system targets only hosts created inside an isolated Mininet virtual network on a single VM under our own control, with no path to any real external host or network. A systematic self-review of this codebase found and fixed 21 issues, two of which would be real security weaknesses in a production deployment of this code: a multi-source leak in

two response handlers, and unsanitized shell interpolation of a source-IP value in two others. Neither reached this paper’s reported evaluation data (verified directly against the harness’s own trial-isolation design and the fact that every source-IP value the harness passes is a registry- or Mininet-derived address, never externally supplied text), and both are documented here alongside what would need fixing before any real deployment, rather than treated as already resolved.

IX. Conclusion

We presented a registry-driven residential IoT defense system comparing three response-selection policies against naive baselines and an external IDS on real, captured Mininet traffic, with a statistical and ablation treatment uncommon in comparable systems papers. Our central positive result is that all three policies substantially and reliably outperform naive fixed-response and no-response baselines, and that policy choice (rule-based vs. game-theoretic vs. learned) matters specifically at decision boundaries rather than on confidently-classified traffic. Our central methodological result is that both of our attempts to strengthen a finding by collecting more evidence instead corrected it: a threefold sample-size increase revised our headline accuracy down by 16 points, and a source-rotation evasion claim did not survive a bug-fixed, higher-powered re-test. We report both corrections in full rather than retaining the more favorable original numbers, and offer this as a concrete argument for treating a scale-up not as a confirmation exercise but as a genuine test the original finding can fail. External validity beyond our single controlled environment remains the primary open problem for future work.

References

- [1] “A realistic experimental comparison of the suricata and snort intrusion-detection systems,” <https://calhoun.nps.edu/server/api/core/bitstreams/6e9ec886-297c-4913-8cc6-80a4c44609a5/content>, naval Postgraduate School.
- [2] “Collaborative device-level botnet detection for internet of things,” <https://www.sciencedirect.com/science/article/pii/S0167404823000822>, scienceDirect.
- [3] “Towards a moving target defense based on stochastic games and honeypots,” <https://www.sciencedirect.com/science/article/pii/S0020025525006206>, scienceDirect.
- [4] “Stackelberg game for resilient collaborative cyber threat detection and response in iot networks,” https://www.researchgate.net/publication/398764870_Stackelberg_Game_for_Resilient_Collaborative_Cyber_Threat_Detection_and_Response_in_IoT_Networks, researchGate.
- [5] “Deep reinforcement learning for intrusion detection in iot: A survey,” <https://arxiv.org/pdf/2405.20038>, arXiv.
- [6] “An optimized reinforcement learning based mtd mutation strategy for securing edge iot against ddos attack,” <https://www.sciencedirect.com/science/article/pii/S2214212625001759>, scienceDirect.